

System Infrastructure and Security Project

C00306572 Callum Matthews

Table Of Contents:

Project Overview:	1
Prometheus:	2
Node Exporter:	6
Lubuntu:	7
Rocky:	9
Prometheus.yml	12
Fail2Ban:	15
Grafana:	20
Fail2Ban Exporter:	22
Lubuntu:	24
Rocky:	25
Alert Manager:	25
Finishing Up Grafana:	28
Security Analysis and Observations:	29
References:	31
Prometheus:	32
Node Exporter:	32
Fail2Ban:	33
Grafana:	33
Fail2Ban Exporter:	33
Alert Manager:	33

Project Overview:

The aim of this project is to set up and configure a monitoring environment across two Linux virtual machines.

This project uses two Linux virtual machines as follows:

- Rocky Linux Server: 192.168.56.10
- Lubuntu Client: 192.168.56.20

In this project, Prometheus and Grafana will be hosted on the Lubuntu client, as we can use the GUI to view the output in a browser using localhost. However, we will scrape metrics and details from the Rocky Server to display here too.

Node Exporter runs on both machines and sends system metrics through port 9100. Prometheus will scrape these metrics at regular intervals. We will use Grafana to connect to Prometheus to display these metrics in an intuitive fashion via a dashboard. Alerts manager will be used to handle our alert message routing when the set thresholds are breached. Finally, Fail2Ban will run on both machines to monitor SSH logins and ban any IP addresses suspected of being involved in a brute force attempt.

All references and resources used will be cited within the document, along with a references page at the end of this report with the full links to each source.

All work completed is my own, and all screenshots provided were taken from my own virtual machines. As there are multiple screenshots, this document has a higher page count than without.

Prometheus:

Prometheus is the core monitoring tool used in this setup. It will scrape metrics from both targets, Ubuntu and Rocky, and will store them as time-series data. Prometheus libraries operate locally, with local storage, so we can set it up to run on the Ubuntu machines and utilise the GUI to view metrics via localhost. (Prometheus, 2026) (Watts, 2025).

To start, I navigated to the Ubuntu Virtual Machine and downloaded the tar file for the Linux version from Prometheus website, version 3.10.0, and extracted its contents to the home directory, as that is where I was currently situated. Once extracted, I had 3 files, prometheus, promtool and the config file prometheus.yml. (Prometheus, 2026) (Watts, 2025) (Cisse, 2025).

Next, I had to move the binaries to the correct folder, as I was in the home directory. /usr/local/bin is the standard file directory for all user installed programs and will allow me to run prometheus from any directory in the terminal. I also had to create configuration directories and move the prometheus files to the /etc/prometheus directory. Prometheus will store its collected data within /var/lib/prometheus. (Watts, 2025). This will act like its database. Commands used are as follows:

- `sudo mv prometheus /usr/local/bin/`
- `sudo mv promtool /usr/local/bin/`
- `sudo mkdir /etc/prometheus`
- `sudo mkdir /var/lib/prometheus`
- `sudo mv prometheus.yml /etc/prometheus/`

The prometheus.yml is the main configuration file. It tells Prometheus which machines to scrape data from, how often and where to send alerts. (Watts, 2025). I will edit this configuration file with more details later, but for the moment I will configure it to scrape Prometheus itself on Port 9090 to test that it is working. To edit the config file, we use nano:

- `sudo nano /etc/prometheus/prometheus.yml`

The basic config to check Prometheus is running is as follows:

scrape_configs:

- job_name: 'prometheus'

static_configs:

- targets: ['localhost:9090']

(Prometheus, No Date)

Before we can verify if this basic configuration is working, we need to complete setting up the Prometheus service.

The next step is to create a prometheus user. This is a security measure. Instead of running Prometheus as root (which is dangerous due to root's highly elevated permissions), I created a dedicated user called prometheus that has no login shell and no home directory. This limits the damage if Prometheus were ever compromised, as it can only access what it needs to. (Watts, 2025). As such, it will appear as a user on the login screen of Ubuntu, like Nagios and then the main labuser account. However, we won't be logging into this user and will remain logged into labuser.

To create the prometheus user, I made a user account called "prometheus" and gave it no home directory nor a shell. (Watts, 2025). If a user attempted to log into the prometheus user, they would be disconnected immediately as they don't have a shell, providing an extra layer of security. I also assigned its ownership over the prometheus folders I created. The -R tag adds recursion, allowing this ownership to apply to all files within these directories. (Watts, 2025).

Commands used:

- `sudo useradd --no-create-home --shell /bin/false prometheus`
- `sudo chown -R prometheus:prometheus /etc/prometheus /var/lib/prometheus`
- `sudo chown prometheus:prometheus /usr/local/bin/prometheus /usr/local/bin/promtool`

(Watts, 2025)

Next, I had to create the system service for prometheus. (Watts, 2025). To do this, I must create a service config file using the following command:

- `sudo nano /etc/systemd/system/prometheus.service`

The contents of the service file are as follows:

[Unit]

Description=Prometheus

Wants=network-online.target

After=network-online.target

[Service]

User=prometheus

Group=prometheus

Type=simple

ExecStart=/usr/local/bin/prometheus \

--config.file=/etc/prometheus/prometheus.yml \

--storage.tsdb.path=/var/lib/prometheus/ \

--web.console.templates=/etc/prometheus/consoles \

--web.console.libraries=/etc/prometheus/console_libraries

[Install]

WantedBy=multi-user.target

Breakdown of Service File for Prometheus:

[Unit] General information about the service:

- Description — human readable name for the service
- Wants/After=network-online.target — tells systemd to wait until the network is available before starting Prometheus

[Service] How to run the service:

- User/Group=prometheus — runs as the dedicated prometheus user we created
- Type=simple — tells systemd this is a straightforward process
- ExecStart — the actual command that starts Prometheus, with four flags:
 - --config.file — points to the prometheus.yml config file
 - --storage.tsdb.path — points to the data storage directory
 - --web.console.templates/libraries — points to the web UI files

[Install]

- WantedBy=multi-user.target — means this service starts when the system boots into normal multi-user mode

(Watts, 2025) (Prometheus, No Date)

To reload and start the service, I issued the following commands:

- sudo systemctl daemon-reload
- sudo systemctl enable prometheus
- sudo systemctl start prometheus

At this point, when I open a browser window and navigate to **http://localhost:9090/targets** I can see prometheus running. However, as I have yet to configure the scrapers for both machines, I can only see prometheus scraping itself on port 9090. This will be rectified when adding node exporters to both machines which will display 3 different jobs, one for Rocky, one for Linux, and one for Prometheus itself.

For clarity, below is the Prometheus.yml config file, with the Node Export Jobs set up. I will cover Node Exporter in the next section, but for the moment we can look at the config file to see what we will end up with. There is default settings preconfigured at the top:

```
Matthews Callum C00306572_Lubuntu_25/26

labuser@ubuntu-Callum: ~
File Actions Edit View Help
labuser@ubuntu-Callum: ~ x
labuser@ubuntu-Callum:~$ cat /etc/prometheus/prometheus.yml
# my global config
global:
  scrape_interval: 15s # Set the scrape interval to every 15 seconds. Default is every 1 minute.
  evaluation_interval: 15s # Evaluate rules every 15 seconds. The default is every 1 minute.
  # scrape_timeout is set to the global default (10s).

# Alertmanager configuration
alerting:
  alertmanagers:
    - static_configs:
        - targets:
            # - alertmanager:9093

# Load rules once and periodically evaluate them according to the global 'evaluation_interval'.
rule_files:
  # - "first_rules.yml"
  # - "second_rules.yml"

# A scrape configuration containing exactly one endpoint to scrape:
# Here it's Prometheus itself.
scrape_configs:
  # The job name is added as a label 'job=<job_name>' to any timeseries scraped from this config.
  - job_name: "prometheus"

    # metrics_path defaults to '/metrics'
    # scheme defaults to 'http'.

    static_configs:
      - targets: ["localhost:9090"]
        # The label name is added as a label 'label_name=<label_value>' to any timeseries scraped from this config.
        labels:
          app: "prometheus"

  # this is the job added for node_exporter, on local host port 9100
  - job_name: "node_exporter_Lubuntu"
    static_configs:
      - targets: ["localhost:9100"]

  # this is the job for node_exporter on Rocky Linux, not local host, using rockys ip add on port 9100
  - job_name: "node_exporter_Rocky"
    static_configs:
      - targets: ["192.168.56.10:9100"]
labuser@ubuntu-Callum:~$
```

Node Exporter:

To scrape metrics from each system to Prometheus, I must set up Node Exporters on both virtual machines. Node Exporter is a prometheus tool that scrapes system metrics from configured machines, such as CPU usage, disk space and network statistics. I have configured them to listen to port 9100 with local host. (Virtana, 2026) (Prometheus, 2026).

Lubuntu:

First, I had to download node exporter. I was able to install it from the GitHub repository using wget from within my home directory and then extracted it using tar (GitHub, 2026). Like before, I moved the extracted files to the /usr/local/bin/ directory. I also created a user in a similar fashion to Prometheus and provided it with no home directory or shell. Then, I had to set up the system service and created a service file like before. This process was extremely easy after having already installed Prometheus. (Chauhan, 2024).

The commands used to do this on Lubuntu are below:

- wget https://github.com/prometheus/node_exporter/releases/download/v1.8.2/node_exporter-1.8.2.linux-amd64.tar.gz
- tar xvf node_exporter-1.8.2.linux-amd64.tar.gz
- sudo mv node_exporter-1.8.2.linux-amd64/node_exporter /usr/local/bin/
- sudo useradd --no-create-home --shell /bin/false node_exporter
- sudo chown node_exporter:node_exporter /usr/local/bin/node_exporter
- sudo nano /etc/systemd/system/node_exporter.service

I created the service file to contain the following:

[Unit]

Description=Node Exporter

Wants=network-online.target

After=network-online.target

[Service]

User=node_exporter

Group=node_exporter

Type=simple

ExecStart=/usr/local/bin/node_exporter

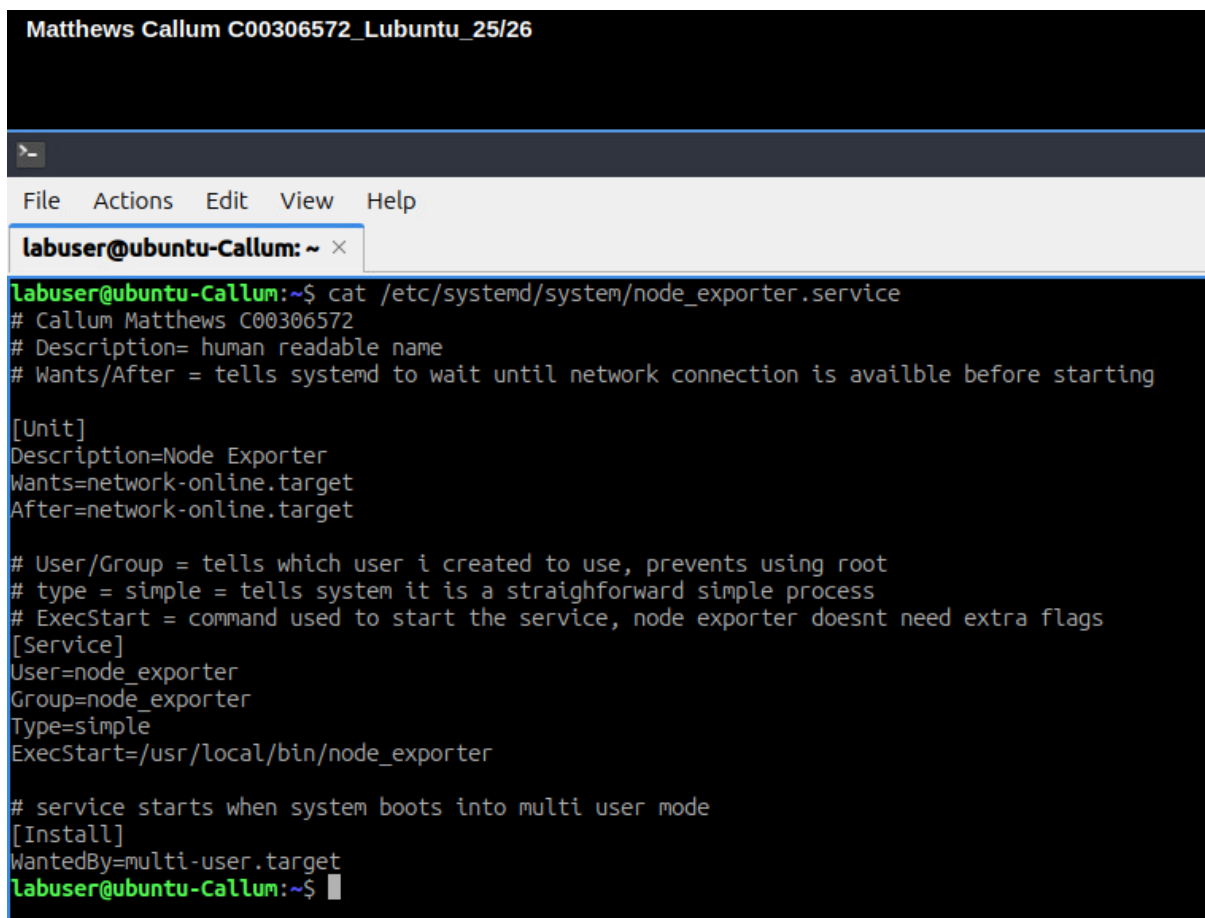
[Install]

WantedBy=multi-user.target

(Chauhan, 2024).

Note that this service build is simpler than Prometheus, as there's only one line in ExecStart with no flags, whereas Prometheus needed four flags pointing to its config file, data directory and web console files. Node Exporter is a simpler tool that just collects and exposes metrics without needing any further configuration.

The following image is the Node Exporter Service File for Ubuntu:



The screenshot shows a terminal window titled "Matthews Callum C00306572_Lubuntu_25/26". The terminal displays the command `cat /etc/systemd/system/node_exporter.service` and its output. The output is the content of the service file, which includes comments, unit settings, service settings, and install settings.

```
labuser@ubuntu-Callum:~$ cat /etc/systemd/system/node_exporter.service
# Callum Matthews C00306572
# Description= human readable name
# Wants/After = tells systemd to wait until network connection is available before starting

[Unit]
Description=Node Exporter
Wants=network-online.target
After=network-online.target

# User/Group = tells which user i created to use, prevents using root
# type = simple = tells system it is a straightforward simple process
# ExecStart = command used to start the service, node exporter doesnt need extra flags
[Service]
User=node_exporter
Group=node_exporter
Type=simple
ExecStart=/usr/local/bin/node_exporter

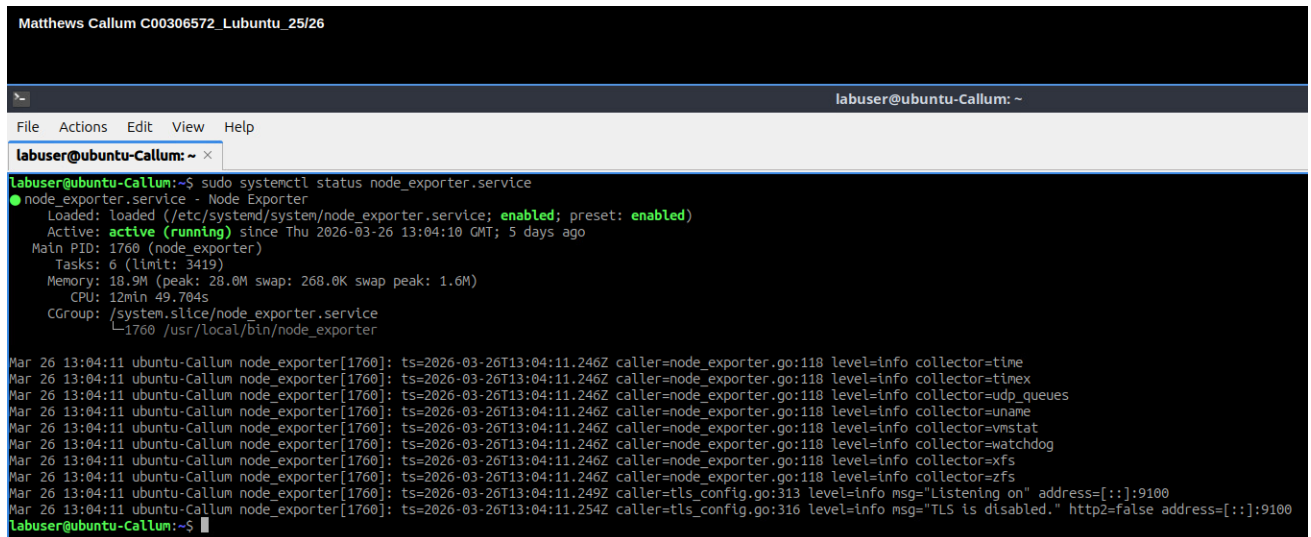
# service starts when system boots into multi user mode
[Install]
WantedBy=multi-user.target
labuser@ubuntu-Callum:~$
```

Finally, I had to start the service, and verify it was running:

- `sudo systemctl daemon-reload`
- `sudo systemctl enable node_exporter`
- `sudo systemctl start node_exporter`
- `sudo systemctl status node_exporter`

Once I set up Node Exporter on Rocky, I will configure the prometheus.yml file to add the node_exporter job for both machines, which will be viewable on Prometheus through <http://localhost:9100/metrics>.

As can be seen in the image below, node exporter is active and running on Ubuntu:



```
Matthews Callum C00306572_Lubuntu_25/26
labuser@ubuntu-Callum: ~
File Actions Edit View Help
labuser@ubuntu-Callum: ~ x
labuser@ubuntu-Callum:~$ sudo systemctl status node_exporter.service
● node_exporter.service - Node Exporter
   Loaded: loaded (/etc/systemd/system/node_exporter.service; enabled; preset: enabled)
   Active: active (running) since Thu 2026-03-26 13:04:10 GMT; 5 days ago
     Main PID: 1760 (node_exporter)
        Tasks: 6 (limit: 3419)
      Memory: 18.9M (peak: 28.0M swap: 268.0K swap peak: 1.6M)
         CPU: 12min 49.704s
       CGroup: /system.slice/node_exporter.service
              └─1760 /usr/local/bin/node_exporter

Mar 26 13:04:11 ubuntu-Callum node_exporter[1760]: ts=2026-03-26T13:04:11.246Z caller=node_exporter.go:118 level=info collector=time
Mar 26 13:04:11 ubuntu-Callum node_exporter[1760]: ts=2026-03-26T13:04:11.246Z caller=node_exporter.go:118 level=info collector=timex
Mar 26 13:04:11 ubuntu-Callum node_exporter[1760]: ts=2026-03-26T13:04:11.246Z caller=node_exporter.go:118 level=info collector=udp_queues
Mar 26 13:04:11 ubuntu-Callum node_exporter[1760]: ts=2026-03-26T13:04:11.246Z caller=node_exporter.go:118 level=info collector=uname
Mar 26 13:04:11 ubuntu-Callum node_exporter[1760]: ts=2026-03-26T13:04:11.246Z caller=node_exporter.go:118 level=info collector=vmstat
Mar 26 13:04:11 ubuntu-Callum node_exporter[1760]: ts=2026-03-26T13:04:11.246Z caller=node_exporter.go:118 level=info collector=watchdog
Mar 26 13:04:11 ubuntu-Callum node_exporter[1760]: ts=2026-03-26T13:04:11.246Z caller=node_exporter.go:118 level=info collector=xfs
Mar 26 13:04:11 ubuntu-Callum node_exporter[1760]: ts=2026-03-26T13:04:11.246Z caller=node_exporter.go:118 level=info collector=zfs
Mar 26 13:04:11 ubuntu-Callum node_exporter[1760]: ts=2026-03-26T13:04:11.249Z caller=tlscfg.go:313 level=info msg="Listening on" address=[::]:9100
Mar 26 13:04:11 ubuntu-Callum node_exporter[1760]: ts=2026-03-26T13:04:11.254Z caller=tlscfg.go:316 level=info msg="TLS is disabled." http2=false address=[::]:9100
labuser@ubuntu-Callum:~$
```

Rocky:

The process on Rocky Linux is very similar to that on Lubuntu. Like on Lubuntu, I used the “wget” command to download node exporter. However, this wasn’t installed on Rocky, so I must install “wget”. A later issue I encountered was SELinux permissions, but I solved this as seen later below. The commands used to install Node Exporter are as follows:

- `sudo dnf install wget -y`
- `wget`
https://github.com/prometheus/node_exporter/releases/download/v1.8.2/node_exporter-1.8.2.linux-amd64.tar.gz
- `tar xvf node_exporter-1.8.2.linux-amd64.tar.gz`
- `sudo mv node_exporter-1.8.2.linux-amd64/node_exporter /usr/local/bin/`
- `sudo useradd --no-create-home --shell /bin/false node_exporter`
- `sudo chown node_exporter:node_exporter /usr/local/bin/node_exporter`
- `sudo nano /etc/systemd/system/node_exporter.service`

The service config on Rocky Linux is the same as was used on Lubuntu:

[Unit]

Description=Node Exporter

Wants=network-online.target

After=network-online.target

[Service]

User=node_exporter

Group=node_exporter

Type=simple

ExecStart=/usr/local/bin/node_exporter

[Install]

WantedBy=multi-user.target

(Chauhan, 2024).

The following image is of the Node Exporter Service File on Rocky Linux:

```

[labuser@rocky-Callum ~]$ cat /etc/systemd/system/node_exporter.service
# Callum Matthews C00306572
# Description: Human Readable name
# Wants/After = tells system to wait for network connection before running
[Unit]
Description=Node Exporter
Wants=network-online.target
After=network-online.target

# Assign user/group we created, called node_exporter
# type = simple single process
# ExecStart command, different to prometheus as no flags
[Service]
User=node_exporter
Group=node_exporter
Type=simple
ExecStart=/usr/local/bin/node_exporter

# start when system starts multi user mode
[Install]
WantedBy=multi-user.target
[labuser@rocky-Callum ~]$

```

Once I had configured the service, I had to start node exporter in the same manner as I did on Ubuntu:

- `sudo systemctl daemon-reload`
- `sudo systemctl enable node_exporter`
- `sudo systemctl start node_exporter`
- `sudo systemctl status node_exporter`

Here I encountered my first issue, what was unique to Rocky Linux. I encountered an error relating to SELinux, which was an error I had seen before when installing Samba on Rocky Linux previously. To fix it, I just had to change the SELinux permission on Rocky, and restart node exporter:

- `sudo restorecon -v /usr/local/bin/node_exporter`
- `sudo systemctl restart node_exporter`
- `sudo systemctl status node_exporter`

SELinux is a security architecture for Linux Systems that allows administrators to have more control over who accesses a system. (RedHat, 2019). I didn't run into this issue with Ubuntu, and that's because SELinux ships with RHEL based systems, such as Rocky Linux. The SELinux permissions only affect system services attempting to run, so if I tried to run Node Exporter from the terminal, in theory it would run fine, but as I want to enable it as a service to feed metrics out automatically as a service, it clashed with SELinux permissions, showing the error: “**code=exited, status=203/EXEC**”. I could have edited the config for SELinux to put it into passive mode to prevent any further errors related to system services, but I chose to just use **restorecon** for node exporter to prevent disabling SELinux altogether, to prevent any reduction in already established security settings. (Informatica, 2023) (RedHat, No Date).

Finally, I needed to open the firewall rule for it to work. I used the following commands to open the port needed, in this case port 9100:

- `sudo firewall-cmd --permanent --add-port=9100/tcp`
- `sudo firewall-cmd --reload`

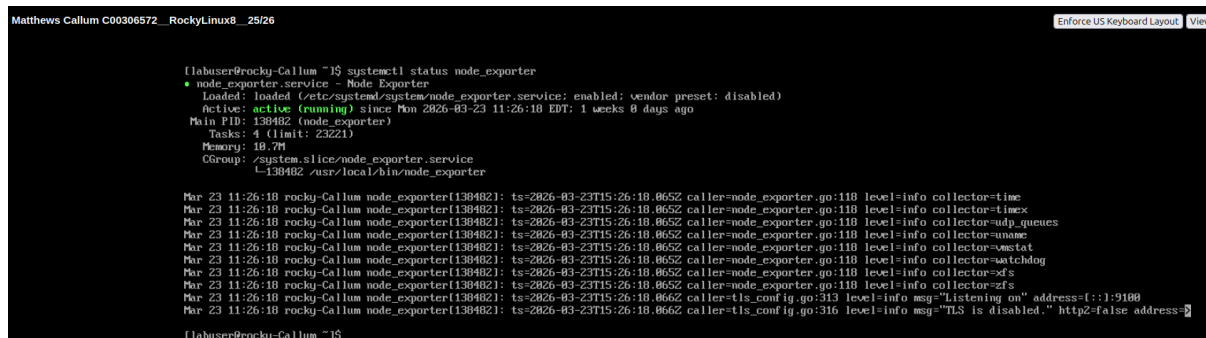
Verify port 9100 is open with:

- `sudo firewall-cmd --list-ports`

The firewall issue I had on Rocky didn't occur on Ubuntu, as Rocky is shipped with “firewalld” enabled by default, causing it to block all incoming connections unless specifically allowed. Hence the need to configure the firewall on Rocky Linux and allow port 9100. If I didn't open the port, we would see Rocky as being “Down” on Prometheus. I encountered this issue after I configured the node exporter and edited the `prometheus.yml` file to display the two virtual machines on Prometheus, but I am highlighting it here as it is a step I should have done first to ensure that there would be no errors later.

Finally, verify that Node Exporter is up and running, as can be seen in the below image:

- `sudo systemctl status node_exporter`



```
Matthews Callum C00306572_RockyLinux8_25/26 Enforce US Keyboard Layout View

[labuser@rocky-Callum ~]$ systemctl status node_exporter
● node_exporter.service - Node Exporter
   Loaded: loaded (/etc/systemd/system/node_exporter.service; enabled; vendor preset: disabled)
   Active: active (running) since Mon 2026-03-23 11:26:18 EDT; 1 weeks 0 days ago
     Main PID: 130482 (node_exporter)
        Tasks: 4 (limit: 23221)
       Memory: 10.7M
      CGroup: /system.slice/node_exporter.service
              └─130482 /usr/local/bin/node_exporter

Mar 23 11:26:18 rocky-Callum node_exporter[130482]: ts=2026-03-23T15:26:18.065Z caller=node_exporter.go:110 level=info collector=time
Mar 23 11:26:18 rocky-Callum node_exporter[130482]: ts=2026-03-23T15:26:18.065Z caller=node_exporter.go:110 level=info collector=timeex
Mar 23 11:26:18 rocky-Callum node_exporter[130482]: ts=2026-03-23T15:26:18.065Z caller=node_exporter.go:110 level=info collector=udp_queues
Mar 23 11:26:18 rocky-Callum node_exporter[130482]: ts=2026-03-23T15:26:18.065Z caller=node_exporter.go:110 level=info collector=uname
Mar 23 11:26:18 rocky-Callum node_exporter[130482]: ts=2026-03-23T15:26:18.065Z caller=node_exporter.go:110 level=info collector=vmstat
Mar 23 11:26:18 rocky-Callum node_exporter[130482]: ts=2026-03-23T15:26:18.065Z caller=node_exporter.go:110 level=info collector=watchdog
Mar 23 11:26:18 rocky-Callum node_exporter[130482]: ts=2026-03-23T15:26:18.065Z caller=node_exporter.go:110 level=info collector=xfs
Mar 23 11:26:18 rocky-Callum node_exporter[130482]: ts=2026-03-23T15:26:18.065Z caller=node_exporter.go:110 level=info collector=zfs
Mar 23 11:26:18 rocky-Callum node_exporter[130482]: ts=2026-03-23T15:26:18.066Z caller=tls_config.go:313 level=info msg="Listening on" address=[::]:9100
Mar 23 11:26:18 rocky-Callum node_exporter[130482]: ts=2026-03-23T15:26:18.066Z caller=tls_config.go:316 level=info msg="TLS is disabled." http2=false address=[::]:9100

[labuser@rocky-Callum ~]$
```

The next step, now that Node Exporter is running on both Rocky and Ubuntu, is to configure the `prometheus.yml` file to scrape metrics from both machines. As Prometheus is running on Ubuntu, I can just check on Localhost, but as Rocky is a separate machine, I will need to add its IP address. To verify I had the correct IP address recorded for Rocky, I doubled checked using the “`ip a`” command, which showed 192.168.56.10 as Rocky’s IP address.

Prometheus.yml

Once Node Exporters were set up on both machines, it was time to go back to Lubuntu and edit the prometheus.yml config again file to allow prometheus to read metrics taken from both devices using the Node Exporters. To do this, I edited the prometheus.yml config file using nano.

Prometheus uses the prometheus.yml file as its main configuration file. It controls who to scrape, how often to scrape and where to send alerts when a rule is triggered.

In the configuration, you set who to scrape under scrape_configs:

At this point after configuring Node Exporter, we should have three jobs, one for Prometheus itself, one for Lubuntu and one for Rocky. Prometheus will be checking its own metrics on port 9090 as covered previously, while it will check Lubuntu and Rocky through port 9100.

Prometheus is running on the Lubuntu Virtual Machine, so when it scrapes Lubuntu's Node Exporter it can just refer to itself using localhost.

Rocky Linux, however, is a separate machine on the network, so Prometheus needs to know its actual IP address (which is 192.168.56.10) to reach it over the network. Using localhost for Rocky would just point back to Lubuntu itself and never reach Rocky.

To add these jobs, we edit the file with nano again:

- `sudo nano /etc/prometheus/prometheus.yml`

Add the following jobs for Lubuntu and Rocky node exporters to preexisting config:

##Previously added:

scrape_configs:

- job_name: 'prometheus'

static_configs:

- targets: ['localhost:9090']

##New lines added below:

- job_name: 'node_exporter_lubuntu'

static_configs:

- targets: ['localhost:9100']

- job_name: 'node_exporter_rocky'

static_configs:

```
- targets: ['192.168.56.10:9100']
```

(Prometheus, No Date)

We set a job name for each exporter and point to where the metrics will come from by specifying targets under static_configs. The current prometheus.yml file is as follows, note I have shown this previously under the Prometheus section for clarity:

```
Matthews Callum C00306572_Lubuntu_25/26
labuser@
File Actions Edit View Help
labuser@ubuntu-Callum: ~ x
labuser@ubuntu-Callum:~$ cat /etc/prometheus/prometheus.yml
# my global config
global:
  scrape_interval: 15s # Set the scrape interval to every 15 seconds. Default is every 1 minute.
  evaluation_interval: 15s # Evaluate rules every 15 seconds. The default is every 1 minute.
  # scrape_timeout is set to the global default (10s).

# Alertmanager configuration
alerting:
  alertmanagers:
    - static_configs:
      - targets:
        # - alertmanager:9093

# Load rules once and periodically evaluate them according to the global 'evaluation_interval'.
rule_files:
  # - "first_rules.yml"
  # - "second_rules.yml"

# A scrape configuration containing exactly one endpoint to scrape:
# Here it's Prometheus itself.
scrape_configs:
  # The job name is added as a label `job=<job_name>` to any timeseries scraped from this config.
  - job_name: "prometheus"

    # metrics_path defaults to '/metrics'
    # scheme defaults to 'http'.

    static_configs:
      - targets: ["localhost:9090"]
        # The label name is added as a label `label_name=<label_value>` to any timeseries scraped from this config.
        labels:
          app: "prometheus"

# this is the job added for node_exporter, on local host port 9100
- job_name: "node_exporter_Lubuntu"
  static_configs:
    - targets: ["localhost:9100"]

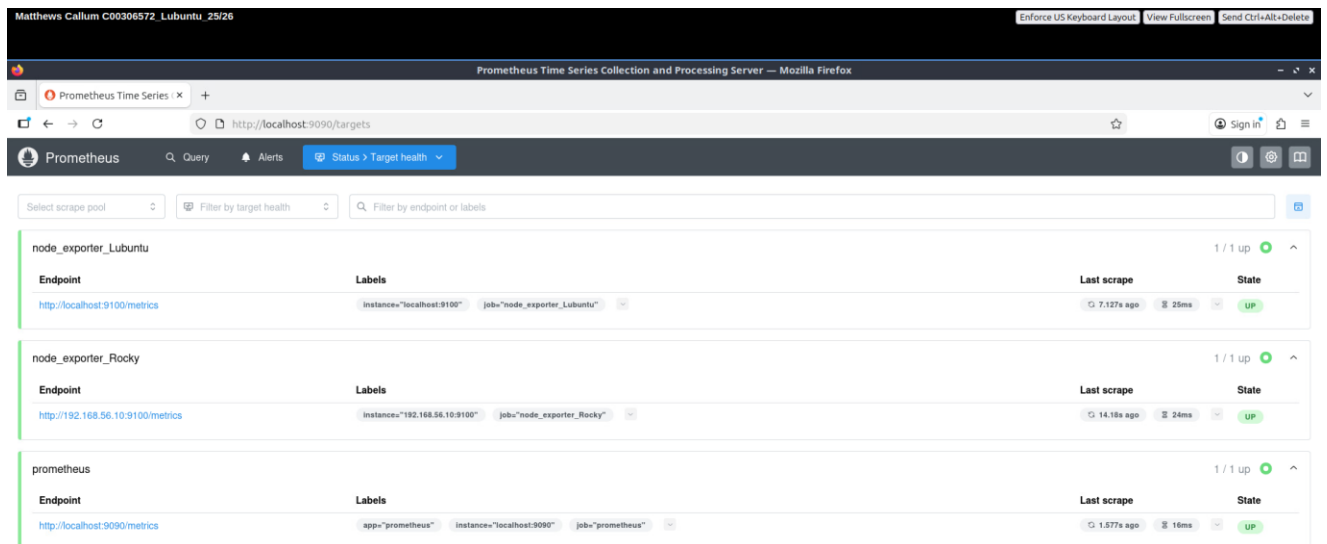
# this is the job for node_exporter on Rocky Linux, not local host, using rockys ip add on port 9100
- job_name: "node_exporter_Rocky"
  static_configs:
    - targets: ["192.168.56.10:9100"]
labuser@ubuntu-Callum:~$
```

Then, reload Prometheus with:

- `sudo systemctl restart prometheus`

Check both node exporters appear on Prometheus: <http://localhost:9090/targets>

As can be seen below, Prometheus is showing both Rocky and Lubuntu with a status of **UP**. The page also shows the last scrape time and scrape duration for each target, confirming data is being collected in real time.



The screenshot shows the Prometheus web interface at <http://localhost:9090/targets>. The interface displays a list of targets with their endpoints, labels, last scrape time, and state. All three targets are in the 'UP' state.

Target	Endpoint	Labels	Last scrape	State
node_exporter_Lubuntu	http://localhost:9100/metrics	instance="localhost:9100" job="node_exporter_Lubuntu"	7.127s ago 25ms	UP
node_exporter_Rocky	http://192.168.56.10:9100/metrics	instance="192.168.56.10:9100" job="node_exporter_Rocky"	14.18s ago 24ms	UP
prometheus	http://localhost:9090/metrics	app="prometheus" instance="localhost:9090" job="prometheus"	1.577s ago 16ms	UP

Fail2Ban:

Fail2Ban protects Linux servers from brute force attacks by monitoring log files and automatically banning IP addresses that show malicious behaviour based off a predefined set of rules established by an administrator. (LinuxCapable, 2026).

I have configured Fail2Ban on both the Ubuntu and Rocky Linux virtual machines to monitor for failed SSH login attempts. Fail2Ban allows you to configure a “Jail” where you can set requirements to ban an IP address, based on the amount of failed login attempts over a predefined period. (DigitalOcean, 2022).

The Jail file acts as a config for Fail2Ban, and I configured one for each virtual machine. These files contain two sets of rules, Default and SSHD. The default block is a blanket set of rules, that establish a ban time, along with a time interval to check for failed login attempts. These rules can then be overwritten by each subset, like SSH logins for example. I configured both files across the two virtual machines the same.

To do this, I first installed fail2ban on Rocky using the Epel Repository, as Fail2Ban is not available through default repositories. Once installed, I could start the service and check that it is running. Once running, we can set the configuration details in the Jail file. (LinuxCapable, 2026).

The commands used on Rocky Linux are as follows:

- `sudo dnf install epel-release -y`
- `sudo dnf install fail2ban -y`
- `sudo systemctl enable fail2ban`
- `sudo systemctl start fail2ban`
- `sudo nano /etc/fail2ban/jail.local`
- `sudo systemctl restart fail2ban`
- `sudo fail2ban-client status sshd`

The jail.local file was configured on both VMs as follows:

[DEFAULT]

`bantime = 10m`

`findtime = 10m`

`maxretry = 3`

`backend = systemd`

`ignoreip = 192.168.56.10 192.168.56.20`

[sshd]

enabled = true

port = ssh

logpath = %(sshd_log)s

bantime = 1h

findtime = 10m

Note, the actual file on the virtual machines contains comments.

The default set of rules is a blanket term that applies to all of Fail2Ban rules, and any subset such as [sshd] can have their own rules that will overwrite the default set within their own class.

Bantime is the set amount of time an IP address stays banned for. The default is 10 minutes. For SSH, I increased this to 1 hour. Max attempts are the maximum number of times a login attempt can fail before resulting in a ban, and the findtime is the window in which Fail2Ban looks back in the log files. I have set the config to ignore the IP address of both Rocky and Ubuntu Virtual machines so that I don't accidentally ban myself in the event of a failed SSH attempt, but this can be removed. (LinuxCapable, 2026).

Finally, backend and logpath allows you to specify where Fail2Ban is to search for logs regarding failed login attempts.

To install on Ubuntu, I repeated the same steps, just slightly varied to allow for the use of different package managers across distributions. The config file remains the same across both VM's. (DigitalOcean, 2022).

- sudo apt install fail2ban -y
- sudo systemctl enable fail2ban
- sudo systemctl start fail2ban
- sudo nano /etc/fail2ban/jail.local
- sudo systemctl restart fail2ban
- sudo fail2ban-client status sshd

The jail.local config files from both machines are shown below.

Rocky Linux:

```
Matthews Callum C00306572_RockyLinux8_25/26

[labuser@rocky-Callum ~]$ sudo cat /etc/fail2ban/jail.local
[sudo] password for labuser:
# Callum Matthews C00306572
# Rocky Linux Jail Config

# Default Rules, any extra set of rules below will overwrite these in relation to what they are checking for.
# bantime = amount of time an ip address stays banded. Time denoted as m for minutes, h for hours
# findtime = how far back in time into logs we check for other failed attempts
# maxretry = max attempts allowed for failed logins before ban
# backend = tells fail2ban to read logs from systemd

[DEFAULT]
bantime = 10m
findtime = 10m
maxretry = 3
backend = systemd

# ip addresses i dont want banded, so the actual ip of lubuntu and rock
ignoreip = 192.168.50.10 192.168.50.20

# SSH rules, will overwrite any time defaults above.
# enabled = true activates service
# port = ssh to specifiy ssh connections
# logpath = where fail2ban to check for failed attempt logs

[sshd]
enabled = true
port = ssh
logpath = %(sshd_log)s
bantime = 1h
findtime = 10m
[labuser@rocky-Callum ~]$ _
```

Lubuntu:

```
Matthews Callum C00306572_Lubuntu_25/26

labuser@ubuntu-Callum: ~
File Actions Edit View Help
labuser@ubuntu-Callum: ~ x
labuser@ubuntu-Callum:~$ sudo cat /etc/fail2ban/jail.local
#Callum Matthews C00306572
# Lubuntu Fail2Ban Jail Config

# Default rules, any extra set of rule below will overwrite these in realtion to what they are checking
# bantime is amount of time an ip stays banned for
# find time is how far back into logs we check for failed ips over set interval
# max try is max amount of attmpts within findtime of log file window
# backend tells fail2ban to read logs from system journal

[DEFAULT]
bantime = 10m
findtime = 10m
maxretry = 3
backend = systemd

# ip addresses i dont want bannded, lubuntu and rocky
ignoreip = 192.168.56.10 192.168.56.20

#SSH rules, enable, set logpath and ban and find time what will overwrite defaults above
# enabled set to true to activate this jail for ssh
# port for ssh to check for ssh logins
# logpath is where fail2ban checks for failed attempt logs
[sshd]
enabled = true
port = ssh
logpath = %(sshd_log)s
bantime = 1h
findtime = 10m
labuser@ubuntu-Callum:~$
```

To test that Fail2Ban was working, I attempted to log into the Rocky VM using SSH from the Lubuntu VM. I typed the password incorrectly 3 times and could see in the Rocky Terminal that the IP address of Lubuntu was banned. Before doing this, I should note I did remove the IP address of Lubuntu from the ignored IPs section in the jail.local file, and then re added it after verifying it worked. As can be seen in the following image, I

attempted to log into the Rocky Linux Server via SSH from the Lubuntu server., and I entered the password incorrectly 3 time to result in a ban:

```
Matthews Callum C00306572_Lubuntu_25/26

labuser@ubuntu-Callum: ~
File Actions Edit View Help
labuser@ubuntu-Callum: ~ x
labuser@ubuntu-Callum:/etc/prometheus$ ssh labuser@192.168.50.10
wrong pass
wrong padd
wrng
^C
labuser@ubuntu-Callum:/etc/prometheus$ ssh labuser@192.168.50.10
^C
labuser@ubuntu-Callum:/etc/prometheus$ cd ~
labuser@ubuntu-Callum:~$ ssh labuser@192.168.50.10
^C
labuser@ubuntu-Callum:~$ ssh labuser@192.168.56.10
The authenticity of host '192.168.56.10 (192.168.56.10)' can't be established.
ED25519 key fingerprint is SHA256:Z5XWnz8E0U4DD51WlQuLx/yCLjNvLmxeecU0lBl1P6Q.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.56.10' (ED25519) to the list of known hosts.
labuser@192.168.56.10's password:
Permission denied, please try again.
labuser@192.168.56.10's password:
Permission denied, please try again.
labuser@192.168.56.10's password:
labuser@192.168.56.10: Permission denied (publickey,gssapi-keyex,gssapi-with-mic,password).
labuser@ubuntu-Callum:~$
```

By checking the report again, using `sudo fail2ban-client status sshd`, I could see that the total bans were set to 1, and the IP address of Lubuntu was displayed.

```
Matthews Callum C00306572_RockyLinux8_25/26

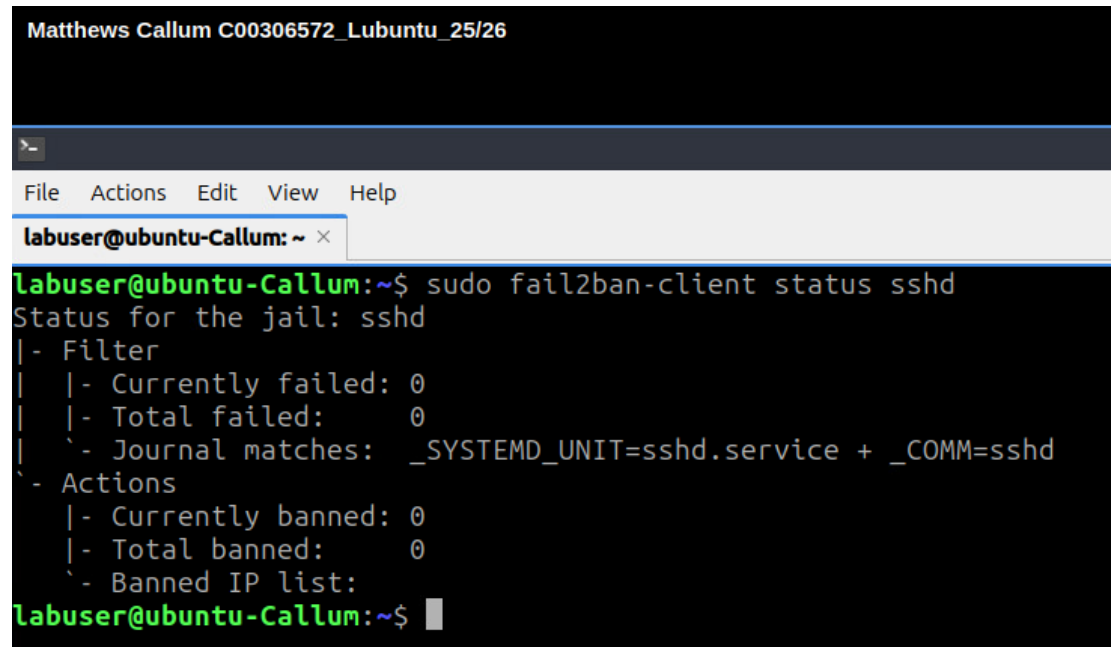
labuser@rocky-Callum ~$ sudo systemctl restart fail2ban
[sudo] password for labuser:
Sorry, try again.
[sudo] password for labuser:
labuser@rocky-Callum ~$ sudo fail2ban-client status sshd
Status for the jail: sshd
|- Filter
| |- Currently failed: 0
| |- Total failed: 0
| `-- Journal matches: _SYSTEMD_UNIT=sshd.service + _COMM=sshd
`- Actions
   |- Currently banned: 0
   |- Total banned: 0
   `-- Banned IP list:
labuser@rocky-Callum ~$ sudo tail -f /var/log/fail2ban.log
[sudo] password for labuser:
2026-03-30 08:23:45.933 fail2ban.jail [1436861]: INFO Jail 'sshd' uses systemd {}
2026-03-30 08:23:45.934 fail2ban.jail [1436861]: INFO Initiated 'systemd' backend
2026-03-30 08:23:45.935 fail2ban.filter [1436861]: INFO maxLines: 1
2026-03-30 08:23:45.951 fail2ban.filtersystemd [1436861]: INFO [sshd] Added journal match for: '_SYSTEMD_UNIT=sshd.service + _COMM=sshd'
2026-03-30 08:23:45.951 fail2ban.filter [1436861]: INFO maxRetry: 3
2026-03-30 08:23:45.951 fail2ban.filter [1436861]: INFO findtime: 600
2026-03-30 08:23:45.951 fail2ban.actions [1436861]: INFO banTime: 3600
2026-03-30 08:23:45.952 fail2ban.filter [1436861]: INFO encoding: UTF-8
2026-03-30 08:23:45.953 fail2ban.filtersystemd [1436861]: INFO [sshd] Jail is in operation now (process new journal entries)
2026-03-30 08:23:45.953 fail2ban.jail [1436861]: INFO Jail 'sshd' started
2026-03-30 08:33:43.441 fail2ban.filter [1436861]: INFO [sshd] Found 192.168.56.20 - 2026-03-30 08:33:43
2026-03-30 08:33:58.158 fail2ban.filter [1436861]: INFO [sshd] Found 192.168.56.20 - 2026-03-30 08:33:58
2026-03-30 08:33:55.783 fail2ban.filter [1436861]: INFO [sshd] Found 192.168.56.20 - 2026-03-30 08:33:55
2026-03-30 08:33:55.935 fail2ban.actions [1436861]: NOTICE [sshd] Ban 192.168.56.20
^C
labuser@rocky-Callum ~$ sudo fail2ban-client status sshd
Status for the jail: sshd
|- Filter
| |- Currently failed: 0
| |- Total failed: 3
| `-- Journal matches: _SYSTEMD_UNIT=sshd.service + _COMM=sshd
`- Actions
   |- Currently banned: 1
   |- Total banned: 1
   `-- Banned IP list: 192.168.56.20
labuser@rocky-Callum ~$
```

To unban Lubuntu after testing, I used the following command:

- `sudo fail2ban-client set sshd unbanip 192.168.56.20`

(Server Fault, 2024).

Now I have verified that Fail2Ban is set up and working on both virtual machines.



The screenshot shows a terminal window titled "Matthews Callum C00306572_Lubuntu_25/26". The terminal has a menu bar with "File", "Actions", "Edit", "View", and "Help". A tab is open with the text "labuser@ubuntu-Callum: ~". The command prompt shows "labuser@ubuntu-Callum:~\$ sudo fail2ban-client status sshd". The output of the command is as follows:

```
labuser@ubuntu-Callum:~$ sudo fail2ban-client status sshd
Status for the jail: sshd
|- Filter
| |- Currently failed: 0
| |- Total failed: 0
| `-- Journal matches: _SYSTEMD_UNIT=sshd.service + _COMM=sshd
`- Actions
   |- Currently banned: 0
   |- Total banned: 0
   `-- Banned IP list:
```

Grafana:

The next step is to set up the Grafana Dashboard to visualise our metrics. Grafana is an open-source analytics and interactive visualization platform that allows you to query, visualize, alert, and understand your metrics from various data sources. It is primarily used for monitoring and observability, providing DevOps teams with real-time insights into application performance, infrastructure health, and business metrics through customizable dashboards. (Zesty, No Date).

To do this, I installed Grafana via apt and added the GPG key.

I had to download dependencies that were missing for me to use WGET. I created a directory for the keyrings, but as I wasn't sure if I already had it, I used a `-p` tag to prevent any error messages if it was already there on the system. (Grafana Labs, 2026)

- `sudo apt install -y apt-transport-https software-properties-common wget`
- `sudo mkdir -p /etc/apt/keyrings/`
- `wget -q -O - https://apt.grafana.com/gpg.key | gpg --dearmor | sudo tee /etc/apt/keyrings/grafana.gpg > /dev/null`

The `wget` command uses 2 tags, `-q` is for quiet, just to stop any progress bar, and `-O` is to output the command so that I could pass it to the next command `gpg --dearmor` which converts the GPG key from its downloaded text format into binary format that apt can read and use. Then I pipe that command into the next, which uses `sudo tee` to write the converted binary key to the file `/etc/apt/keyrings/grafana.gpg`. Using `/dev/null` suppresses the terminal to prevent any progress bar or output filling my screen. This is a security measure that tells the system to only trust Grafana packages signed with this specific key, preventing installation of malicious packages. (Grafana Labs, 2026)

- `echo "deb [signed-by=/etc/apt/keyrings/grafana.gpg] https://apt.grafana.com stable main" | sudo tee /etc/apt/sources.list.d/grafana.list`

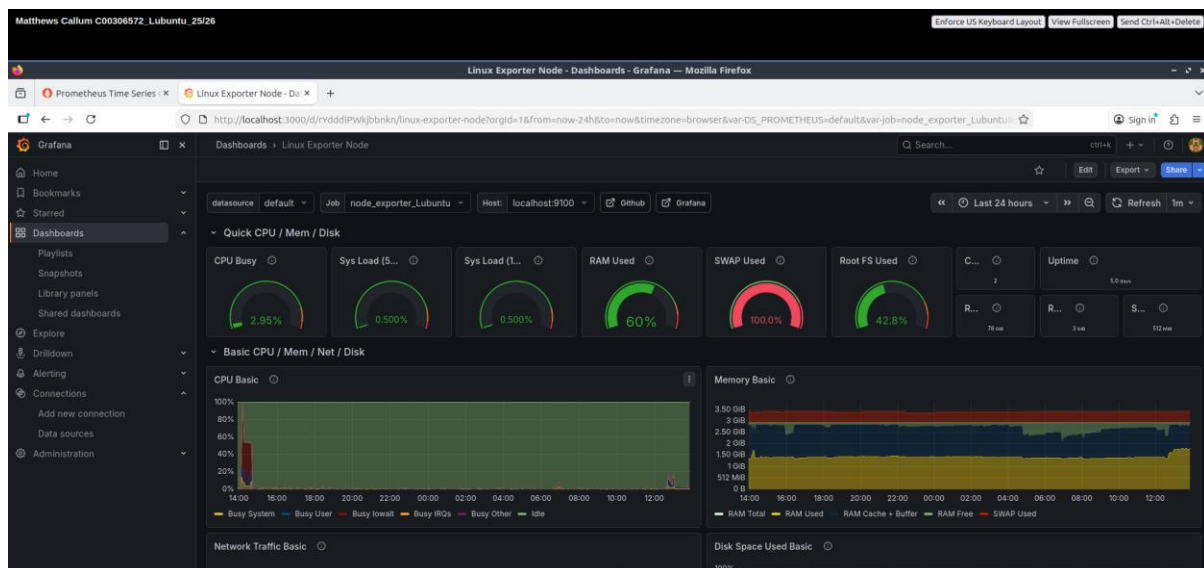
This adds the official Grafana repository to the system's list of software sources, so apt knows where to find and download Grafana from. (Grafana Labs, 2026)

- `sudo apt update`
- `sudo apt install grafana -y`
- `sudo systemctl enable grafana-server`
- `sudo systemctl start grafana-server`
- `sudo systemctl status grafana-server`

To access the Grafana Dashboard, I navigated to: `http://localhost:3000` through Firefox on Ubuntu. I was presented with the Grafana Login screen, and had to input the default username and password, both of which were admin. Then I changed my password to be

labuser, to keep it simple and memorable. Next, I had to select the Prometheus data source from the Data Sources window using the URL: **http://localhost:9090**. I imported the pre-built Node Exporter Full dashboard using template ID **14513** from Grafana's dashboard library, as suggested in the project specification. This provided a fully populated dashboard showing CPU, memory, disk and network metrics in real time.

From here I can see the below dashboard. I can select from the two jobs I previously set up with Node Exporter, one for Ubuntu and one for Rocky, using the drop-down box:



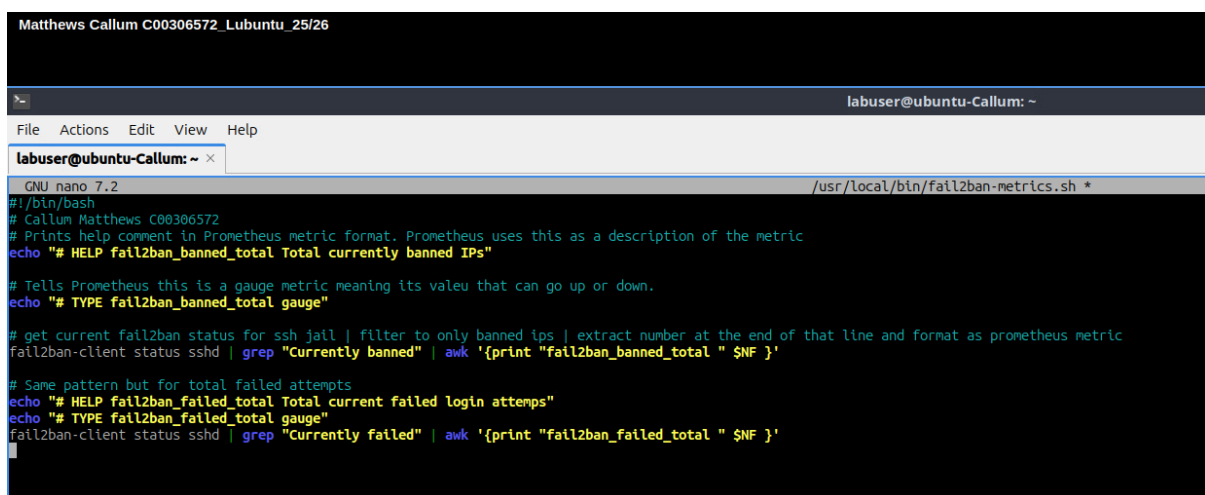
Fail2Ban Exporter:

Once I had the basics set up, it was time to set up an exporter for Fail2ban so that Prometheus could scrape its metrics and visualise them in Grafana. I had to do this on both Rocky and Ubuntu. There were plenty of Git repositories that feature scripts in Python and Go to do this, but a lot of them haven't been maintained in years. Instead, I opted to use the built-in feature of Node Exporter to scrape my fail2ban metrics, called a text file collector. It reads **.prom** files from a specific directory and exposes their metrics to Prometheus. (Prometheus, No Date)

First, I created a text file collector directory. Then I created a bash script that queries Fail2ban and formats the output as Prometheus metrics. To run this automatically, I set up a cron job to execute the script every minute. A cron job is a scheduled task in Linux that runs automatically at a specified time interval. Once the cron job was set, I could update the Node Exporter service file to point to the text file collector directory.

The text file collector directory is: **/var/lib/node_exporter/textfile_collector**

The bash script used was as follows:



```
Matthews Callum C00306572_Lubuntu_25/26
labuser@ubuntu-Callum: ~
File Actions Edit View Help
labuser@ubuntu-Callum: ~ ×
GNU nano 7.2 /usr/local/bin/fail2ban-metrics.sh *
#!/bin/bash
# Callum Matthews C00306572
# Prints help comment in Prometheus metric format. Prometheus uses this as a description of the metric
echo "# HELP fail2ban_banned_total Total currently banned IPs"
# Tells Prometheus this is a gauge metric meaning its value that can go up or down.
echo "# TYPE fail2ban_banned_total gauge"
# get current fail2ban status for sshd | filter to only banned ips | extract number at the end of that line and format as prometheus metric
fail2ban-client status sshd | grep "Currently banned" | awk '{print "fail2ban_banned_total " $NF}'
# Same pattern but for total failed attempts
echo "# HELP fail2ban_failed_total Total current failed login attempts"
echo "# TYPE fail2ban_failed_total gauge"
fail2ban-client status sshd | grep "Currently failed" | awk '{print "fail2ban_failed_total " $NF}'
```

The Cron job is set to run every minute:

```
Matthews Callum C00306572_Lubuntu_25/26

labuser@ubuntu-Callum: ~
File Actions Edit View Help
labuser@ubuntu-Callum: ~ ×
GNU nano 7.2 /tmp/crontab.i7riVN/crontab *
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
# m h dom mon dow   command
# for fail2ban exporter, every minute script runs and outputs to fail2ban.prom for node exporter to pick up automatically
* * * * * /usr/local/bin/fail2ban-metrics.sh > /var/lib/node_exporter/textfile_collector/fail2ban.prom
```

Finally, the Node Exporter Service file was edited to change the ExecStart file path:

```
Matthews Callum C00306572_Lubuntu_25/26

labuser@ubuntu-Callum: ~
File Actions Edit View Help
labuser@ubuntu-Callum: ~ ×
GNU nano 7.2 /etc/systemd/system/node_exporter.se
# Callum Matthews C00306572
# Description= human readable name
# Wants/After = tells systemd to wait until network connection is available before starting

[Unit]
Description=Node Exporter
Wants=network-online.target
After=network-online.target

# User/Group = tells which user i created to use, prevents using root
# type = simple = tells system it is a straightforward simple process
# ExecStart = command used to start the service, node exporter doesnt need extra flags.
# For fail2ban, we add to execstart to tell node exporter where to look for textfile directory for metrics
[Service]
User=node_exporter
Group=node_exporter
Type=simple
ExecStart=/usr/local/bin/node_exporter --collector.textfile.directory=/var/lib/node_exporter/textfile_collector

# service starts when system boots into multi user mode
[Install]
WantedBy=multi-user.target
```

Lubuntu:

On Lubuntu, the commands I used were as follows:

- `sudo mkdir -p /var/lib/node_exporter/textfile_collector`
- `sudo nano /usr/local/bin/fail2ban-metrics.sh`
- `sudo chmod +x /usr/local/bin/fail2ban-metrics.sh`
- `sudo /usr/local/bin/fail2ban-metrics.sh`
- `sudo crontab -e`

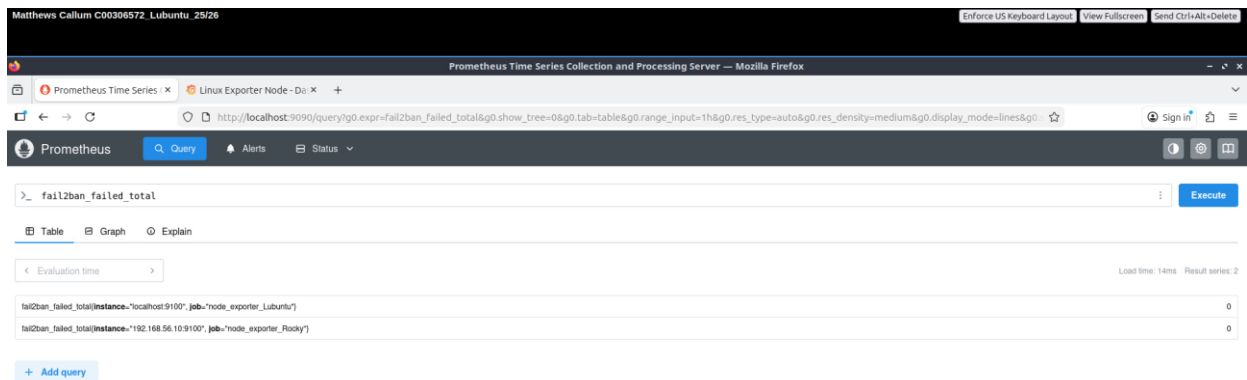
- `sudo systemctl daemon-reload`
- `sudo systemctl restart node_exporter`
- `sudo systemctl status node_exporter`

Rocky:

On Rocky, the commands I used were as follows:

- `sudo mkdir -p /var/lib/node_exporter/textfile_collector`
- `sudo nano /usr/local/bin/fail2ban-metrics.sh`
- `sudo chmod +x /usr/local/bin/fail2ban-metrics.sh`
- `sudo /usr/local/bin/fail2ban-metrics.sh`
- `sudo crontab -e`
- `sudo systemctl daemon-reload`
- `sudo systemctl restart node_exporter`
- `sudo systemctl status node_exporter`

As can be seen, both exporters are viewable on Prometheus:



Alert Manager:

Next, I set up alert manager. I chose to do this via email. As such, I would need a discord server. If CPU usage was to spike, or if there were failed log in attempts on my servers via SSH, I would receive a message in discord. I chose to use discord as email alerts require an app password. (Volz, 2022).

On Lubuntu, I used wget to get the alert manager file from the GitHub repository:
<https://github.com/prometheus/alertmanager> (Prometheus, No Date)

Next, I extracted the files and moved the binaries to the appropriate folders. Then, like with Prometheus, I created a user with no home directory and a shell of /bin/false, and then set the permissions to own the alert manager files.

- wget
`https://github.com/prometheus/alertmanager/releases/download/v0.27.0/alertmanager-0.27.0.linux-amd64.tar.gz`
- `tar xvf alertmanager-0.27.0.linux-amd64.tar.gz`
- `sudo mv alertmanager-0.27.0.linux-amd64/alertmanager /usr/local/bin/`
- `sudo mv alertmanager-0.27.0.linux-amd64/amtool /usr/local/bin/`
- `sudo mkdir /etc/alertmanager`
- `sudo mv alertmanager-0.27.0.linux-amd64/alertmanager.yml /etc/alertmanager/`
- `sudo useradd --no-create-home --shell /bin/false alertmanager`
- `sudo chown -R alertmanager:alertmanager /etc/alertmanager`
- `sudo chown alertmanager:alertmanager /usr/local/bin/alertmanager /usr/local/bin/amtool`

Next, I had to configure the alertmanager.yml file with my newly created discord server and create a webhook URL to send me messages when an alert is created. (Volz, 2022).

- `sudo nano /etc/alertmanager/alertmanager.yml`

To test the alert manager, I opened a terminal in Lubuntu and attempted multiple failed logins attempts to Rocky via SSH. The following screenshots show the alert appearing through alert manager after a few minutes:

- `http://localhost:9090/alerts` — shows **FIRING**

Matthews Callum C00306572 - Ubuntu 25/26 Enforce US Keyboard Layout View Fullscreen Send Ctrl+Alt+Delete

Prometheus Time Series Collection and Processing Server — Mozilla Firefox

Prometheus Time Series × Amount of Currently Ban × Prometheus Time Series × Alertmanager × +

← → ↻ http://localhost:9090/alerts

Prometheus Query Alerts Status

Filter by rule group state Filter by rule name or labels

security_alerts /etc/prometheus/alert_rules.yml FIRING (1) INACTIVE (2)

HighCPUUsage

HighMemoryUsage

Fail2BanBannedIP FIRING (1)

fail2ban_banned_total > 0

for: 1m

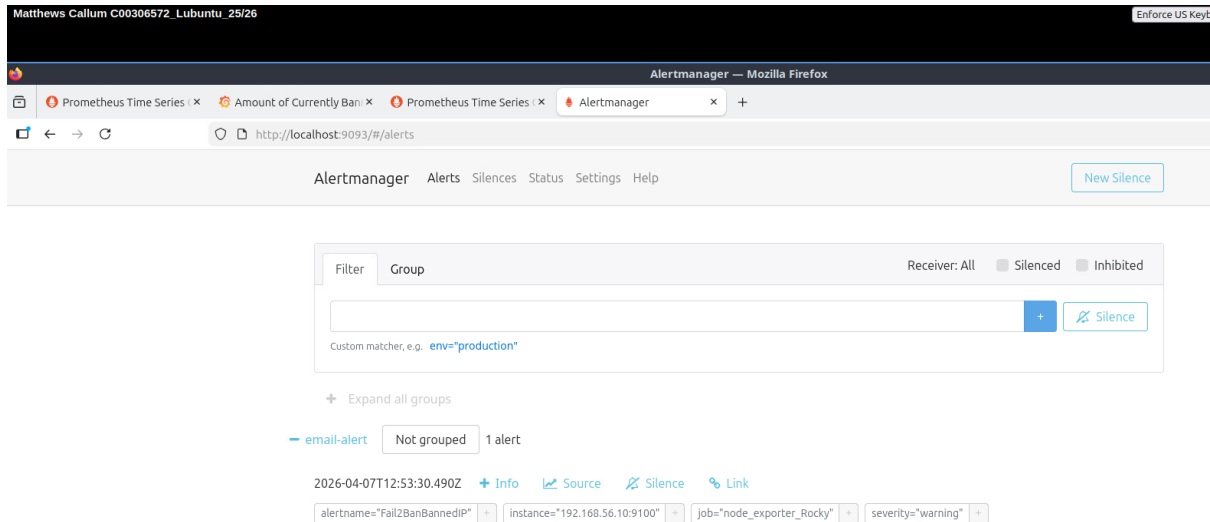
severity: "warning"

description An IP has been banned on {{ \$labels.instance }}

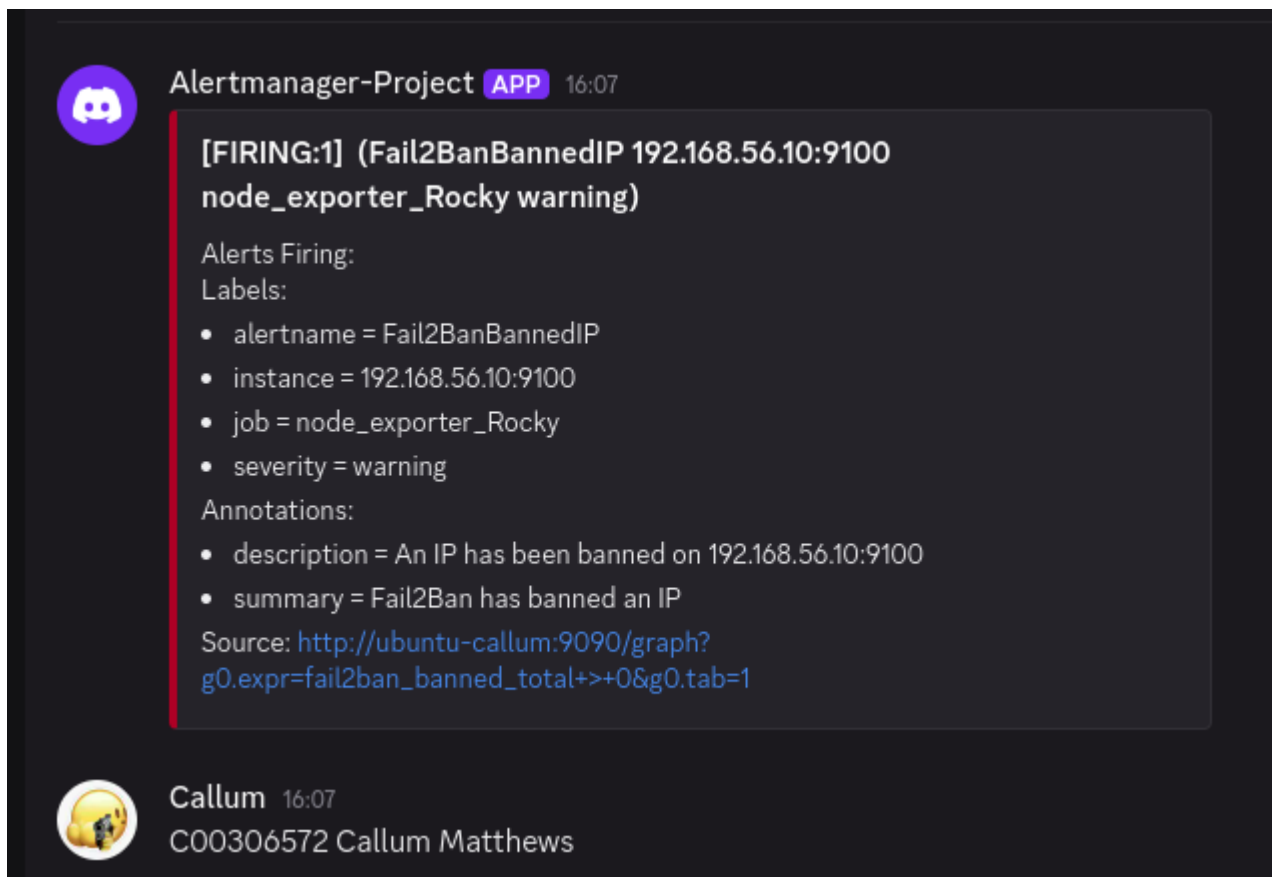
summary Fail2Ban has banned an IP

Alert labels	State	Active Since	Value
alertname="Fail2BanBannedIP" instance="192.168.56.10-9100" job="node_exporter_Rocky" severity="warning"	FIRING	5m 37.088s	1

- <http://localhost:9093> — the alert appears:



- Check Discord, alert appears here after a few minutes:



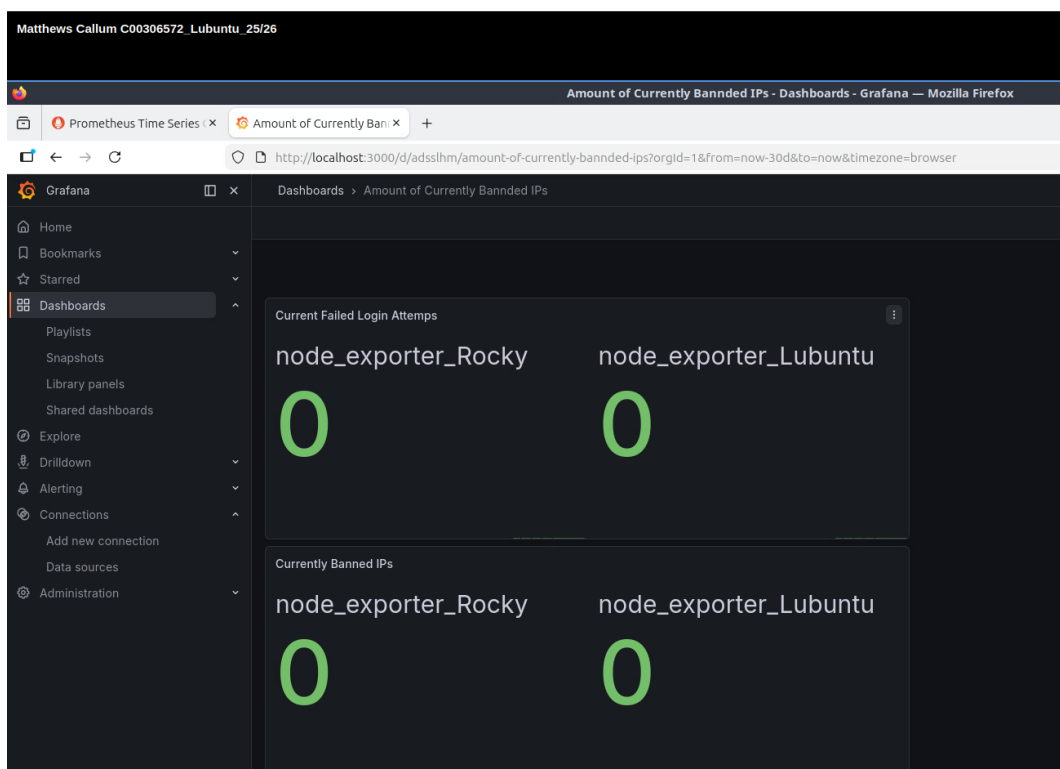
Finishing Up Grafana:

To finish the project, I will add the second dashboard on Grafana in the same manner that I did with the first. This dashboard shows the Currently Banned IP's and the Current Count of Failed attempts from SSH logins, using the Fail2Ban exporter.

To do this, I created a new dashboard and added two stat panels for each virtual machine using the jobs created from the Fail2Ban exporter: fail2ban_banned_total and fail2ban_failed_total. I called these Currently Banned IP's and Current Failed Login Attempts respectively.

I changed the legend field in the query options at the bottom of the panel editor, selecting Custom and typing {{job}} so that it would display the job title of each exporter, so I could differentiate each one by VM name.

I set the visualisation type to "Stat" so that I could display a number, and there is a graph included under this number. The data from this comes from the text file collector I set up earlier. To test that it worked, I attempted 3 failed login attempts through SSH and it changed the number on the dashboard to reflect failed login attempts. Once an IP address gets banned, the Banned IP counter increases, which resets the failed counter back to 0 as is the normal behaviour of Fail2Ban. The below screenshot shows this new dashboard:



Security Analysis and Observations:

After completing this project, there are some strengths and limitations that I have noticed throughout my setup. Firstly, I will outline some of the strengths that this setup has in regard to general system security.

Fail2Ban can detect brute force attempts through SSH, and it allows me to automatically ban offending IP addresses once a breach is detected. This helps to cut down the response time, as without Fail2Ban, I would have to manually detect and respond to potential brute force attempts.

By including an alert manager via Discord, I can get real time updates regards the strength and security of my systems. For example, if an IP address was to be detected to have been involved in a brute force attempt, I would get a message on my phone alerting me that there was an attempted breach. As I can also receive alerts regarding high CPU and memory usage, I am able to be updated of any spikes in usage, which may alert me to an attempted attack or any running issues occurring on my system.

I am also able to visualise my system statistic through Grafana, allowing me to have real time monitoring through my web browser.

Finally, by running services as dedicated users, as opposed to Root, I am following the principle of least privilege. This allows me to minimise the damage that could be done if a service was to be compromised.

However, there are some limitations with my system set up. The first major limitation or security flaw would be the single point of failure that I am operating from. As I am running Grafana, Prometheus and Alert Manager on the Lubuntu virtual machine, if Lubuntu was to go offline, I am left without any system monitoring tools across both VMs.

My Text file Collector for Fail2Ban runs every minute. This means that there is a delay in events being reported. While one minute is not a massive loss of time, in the event of an attack, it might be costly. In practice, this means that there is a delay in a Fail2Ban event and it appearing on Grafana.

There is also no encryption in communication between Prometheus and Node Exporter, as they communicate over HTTP. Theoretically, these communications could be intercepted.

Finally, there are some improvements that could be made on my setup. Firstly, I would like to set up a password, like Grafana, on my Prometheus dashboard. Both Prometheus and Alertmanager are accessible without a username or password. Also, as previously mentioned Grafana is accessible over HTTP. If I was do this project again, I would prefer to see if I could only make Grafana accessible over HTTPS.

Also, my Fail2Ban configuration only monitors SSH. I could expand this to monitor other services to improve security across my systems.

Finally, I set the alert thresholds for CPU and memory usage to 80% and above. If I was to monitor the systems for a few days, I could learn the average usage that both VMs have and configure the alert thresholds to be more in line to what may be considered a high usage for each system.

In conclusion, I did enjoy doing this project, and if I were to undertake this task again, I would make the slight changes outlined previously. However, I am happy with how it currently works, and I did enjoy seeing each aspect of this project come together.

References:

All resources and documentation used are referenced below and are separated by topic for readability.

Prometheus:

Prometheus (2026) *Prometheus – Monitoring system & time series database*. Available at: <https://prometheus.io/> (Accessed 19 March 2026).

Prometheus (No Date) *First steps with Prometheus*. Available at: https://prometheus.io/docs/introduction/first_steps/ (Accessed 19 March 2026).

Watts, D. (2025) *Installing Prometheus as a systemd service on Rocky Linux 9.5*, Medium. Available at: <https://medium.com/@wattsdave/installing-prometheus-as-a-systemd-service-on-rocky-linux-9-5-73ec60f70358> (Accessed 19 March 2026).

Cisse, I. (2025) *A complete guide to Prometheus: Monitoring your infrastructure like a pro*, Medium, 30 March. Available at: <https://medium.com/@ibraheemcisse/a-complete-guide-to-prometheus-monitoring-your-infrastructure-like-a-pro-8684cdadc5a2> (Accessed 19 March 2026).

Node Exporter:

Virtana (2026) *What is a Node Exporter?* Available at: <https://www.virtana.com/glossary/what-is-a-node-exporter> (Accessed 23 March 2026).

Prometheus (2026) *Monitoring Linux host metrics with the Node Exporter*. Available at: <https://prometheus.io/docs/guides/node-exporter/> (Accessed 23 March 2026).

GitHub (2026) *node_exporter: Exporter for machine metrics*, Prometheus on GitHub. Available at: https://github.com/prometheus/node_exporter (Accessed 23 March 2026).

Chauhan, A. (2024) *Step-by-step guide to installing Node Exporter on RHEL 8*, Medium, 10 October. Available at: <https://medium.com/devops-playbook/step-by-step-guide-to-installing-node-exporter-on-rhel-8-10-88d54597312f> (Accessed 23 March 2026).

Red Hat (2019) *What is SELinux?*, Red Hat. Available at: <https://www.redhat.com/en/topics/linux/what-is-selinux> (Accessed 23 March 2026).

Informatica (2023) *Knowledge Base article 000206087*, Informatica Knowledge Base. Available at: https://knowledge.informatica.com/s/article/000206087?language=en_US (Accessed 23 March 2026).

Red Hat (No Date) *Changing SELinux states and modes*, Red Hat Enterprise Linux 8 documentation. Available at: https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/8/html/using_selinux/changing-selinux-states-and-modes_using-selinux (Accessed 23 March 2026).

Fail2Ban:

James, J. (2026) *How to install Fail2ban on Rocky Linux*, LinuxCapable. Available at: <https://linuxcapable.com/how-to-install-fail2ban-on-rocky-linux/> (Accessed 26 March 2026).

Garnett, A. (2022) *How to protect SSH with Fail2Ban on Ubuntu 20.04*, DigitalOcean. Available at: <https://www.digitalocean.com/community/tutorials/how-to-protect-ssh-with-fail2ban-on-ubuntu-20-04> (Accessed 26 March 2026).

Server Fault (2024) *How to unban an IP properly with Fail2Ban*. Available at: <https://serverfault.com/questions/285256/how-to-unban-an-ip-properly-with-fail2ban> (Accessed 26 March 2026).

Grafana:

Zesty (No Date) *What is Grafana?* Available at: <https://zesty.co/finops-glossary/grafana/> (Accessed 31 March 2026).

Grafana Labs (No Date) *Install Grafana on Debian or Ubuntu*. Available at: <https://grafana.com/docs/grafana/latest/setup-grafana/installation/debian/> (Accessed 31 March 2026).

Fail2Ban Exporter:

Prometheus (No Date). Node Exporter Textfile Collector. GitHub. Available at: https://github.com/prometheus/node_exporter#textfile-collector (Accessed 31 March 2023)

Alert Manager:

Prometheus (2026) *alertmanager: Prometheus Alertmanager*, GitHub. Available at: <https://github.com/prometheus/alertmanager> (Accessed 31 March 2026).

Volz, J. (2022) *Sending Prometheus alerts to Discord with Alertmanager v0.25.0*, PromLabs Blog, 23 December. Available at: <https://promlabs.com/blog/2022/12/23/sending-prometheus-alerts-to-discord-with-alertmanager-v0-25-0/> (Accessed 7 April 2026).